

Bài 3. (5 điểm) ĂN KHẾ TRẢ VÀNG

Trong truyện cổ tích Ăn khế trả vàng, người em đã làm đúng theo lời chim thần may túi ba gang ra đảo hoang lấy vàng. Trên hòn đảo có tất cả n khối vàng nằm thành hàng dài dọc theo lối đi, khối thứ i có khối lượng a_i . Vốn là người chất phác lại không tham lam nên người em chỉ muốn lấy một số khối vàng liên tiếp rồi nhanh chóng về nhà, tất nhiên là tổng khối lượng không được vượt quá M là khả năng chở của chim thần.

Yêu cầu: Hãy cho biết người em có bao nhiêu cách chọn ra các khối vàng liên tiếp sao cho tổng khối lượng của các khối được chọn không vượt quá M ?

Dữ liệu vào: Cho từ tệp văn bản TRAVANG.INP có dạng:

- Dòng thứ nhất ghi hai số nguyên dương n và M ($0 < n \leq 10^5, 0 < M \leq 10^9$);
- Dòng thứ hai ghi n số nguyên $a_1, a_2, \dots, a_n$ ($0 < a_i \leq 10^9, i = 1..n$).

Kết quả: Ghi ra tệp văn bản TRAVANG.OUT gồm một dòng ghi một số nguyên là số cách chọn các khối vàng liên tiếp sao cho tổng khối lượng của các khối được chọn không vượt quá M .

Ví dụ:

TRAVANG.INP	TRAVANG.OUT
6 10 8 2 4 15 10 9	7

Ràng buộc:

- Có 70% số điểm tương ứng 70% số test có $0 < n \leq 10^2$.
- Có 20% số điểm tương ứng 20% số test có $10^2 < n \leq 10^3$.
- Có 10% số điểm tương ứng 10% số test có $10^3 < n \leq 10^5$.

“ĂN KHẾ TRẢ VÀNG” – ĐẾM CÁC ĐOẠN CON LIÊN TIẾP CÓ TỔNG KHÔNG VƯỢT QUÁ M

Ý tưởng: Dùng kỹ thuật cửa sổ trượt (two pointers)

- ✓ Duyệt từ trái sang phải, với mỗi r (right), ta cố gắng tìm l (left) sao cho đoạn $[l, r]$ thỏa mãn tổng $\leq M$.
- ✓ Khi đoạn $[l, r]$ hợp lệ $\Rightarrow$ tất cả đoạn $[l, r], [l+1, r], \dots, [r, r]$ đều hợp lệ.
- ✓ Có đúng $(r - l + 1)$ đoạn con kết thúc tại vị trí r thỏa mãn điều kiện.

Tại mỗi bước:

- ✓ Cộng $a[r]$ vào tổng
- ✓ Nếu tổng $> M \rightarrow$ tăng l lên đến khi tổng $\leq M$
- ✓ Đếm $(r - l + 1)$ đoạn con kết thúc tại r

Chạy từng bước: N=6; M=10

I	0	1	2	3	4	5
Khoi_vang[i]	8	2	4	15	10	9-r-l

Tong = 0 L=0, dem_cách=0

R=0: Tong=8; dem_cach=1

R=1: Tong =10 ; dem_cach=3

R=2: tong =14; tong =14-8=6; l=1; dem_cach=5

R=3: tong =21; tong=21-2=19; tổng = 19-4=15;

tong=15-15=0; dem_cach = 5+0=5

r=4: Tong =10; dem_cach=5+1=6

r=5: Tong=19; tong=19-10=9; dem_cach=6+1=7

CODE THAM KHẢO

```
1  #include <bits/stdc++.h>
2  #define name "TRAVANG"
3  using namespace std;
4
5  int main()
6  {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9      if(fopen(name".INP", "r"))
10     {
11         freopen(name".INP", "r", stdin);
12         freopen(name".OUT", "w", stdout);
13     }
14     int n; long long M;
15     cin >> n >> M;
16     vector<long long> khoi_vang(n);
17     for (int i = 0; i < n; i++)
18         cin >> khoi_vang[i];
19
20     long long tong = 0;
21     int l = 0;
22     long long dem_cach = 0;
23     for (int r = 0; r < n; r++)
24     {
25         tong += khoi_vang[r];
26         // Trong khi tổng > M, cần giảm đoạn bằng cách tăng trái
27         while (tong > M && l <= r)
28         {
29             tong -= khoi_vang[l];
30             l++;
31         }
32         // Có (phải - trái + 1) đoạn con kết thúc tại phải thỏa mãn
33         dem_cach += (r - l + 1);
34     }
35     cout << dem_cach << endl;
36     return 0;
37 }
```

Kỹ Thuật Sliding Window Trong Lập Trình

1. Giới Thiệu Về Sliding Window

Sliding Window là một kỹ thuật tối ưu hóa thuật toán, thường được sử dụng để giảm độ phức tạp của các bài toán liên quan đến xử lý mảng hoặc chuỗi. Thay vì sử dụng hai vòng lặp lồng nhau để duyệt tất cả các trường hợp có thể, kỹ thuật Sliding Window giúp ta tận dụng thông tin từ lần duyệt trước để tính toán hiệu quả hơn.

2. Khi Nào Dùng Sliding Window?

Kỹ thuật này thường được áp dụng khi bài toán yêu cầu:

- Tìm tổng, trung bình, hoặc một tính chất nào đó của **một dãy con liên tiếp** trong mảng/chuỗi.
- Xác định độ dài tối đa/tối thiểu của một dãy con thỏa mãn điều kiện nào đó.
- Tối ưu hóa các bài toán có cách giải brute-force ($O(n^2)$) xuống ($O(n)$).

3. Các Loại Sliding Window

a) Fixed Window (Cửa sổ cố định)

Đây là dạng cơ bản nhất, khi cửa sổ có kích thước cố định k . Ví dụ: Tìm tổng lớn nhất của một dãy con có độ dài k trong mảng.

Ví dụ: Tìm tổng lớn nhất của một dãy con có độ dài k

```
//Tìm tổng lớn nhất của dãy con có độ dài k
int TongConMax(int A[], int n, int k)
{
    int maxSum = 0, windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += A[i];
    }
    maxSum = windowSum;
    for (int i = k; i < n; i++)
    {
        windowSum += A[i] - A[i - k];
        maxSum = Math.Max(maxSum, windowSum);
    }
    return maxSum;
}
```

b) Dynamic Window (Cửa sổ động)

Dạng này cửa sổ có kích thước thay đổi, thường được sử dụng khi cần tìm dãy con ngắn nhất/dài nhất thỏa mãn một điều kiện nào đó.

Ví dụ: Tìm dãy con ngắn nhất có tổng $\geq M$

```
//Tìm dãy con ngắn nhất có tổng  $\geq M$ 
int MinSubArrayLen(int M, int A[], int n)
{
    int left = 0, sum = 0, minLength = int.MaxValue;
    for (int right = 0; right < n; right++)
    {
        sum += nums[right];
        while (sum  $\geq$  M) {
            minLength = Math.Min(minLength, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }
    return minLength == int.MaxValue ? 0 : minLength;
}
```

4. Khi Nào Không Nên Dùng Sliding Window?

- Khi phần tử không được sắp xếp và yêu cầu truy cập ngẫu nhiên.
- Khi bài toán yêu cầu kiểm tra nhiều tổ hợp không liên tiếp.
- Khi không có một quy tắc cụ thể nào để mở rộng hoặc thu hẹp cửa sổ.

. 5. Kết Luận

Kỹ thuật **Sliding Window** giúp tối ưu hóa đáng kể các bài toán liên quan đến mảng và chuỗi, giảm độ phức tạp từ $O(n^2)$ xuống $O(n)$. Việc hiểu và áp dụng đúng kỹ thuật này sẽ giúp software engineer tối ưu hóa hiệu suất chương trình một cách đáng kể.

